FEU INSTITUTE OF TECHNOLOGY

COLLEGE OF COMPUTER STUDIES AND MULTIMEDIA ARTS

CS0065
(INTELLIGENT SYSTEMS)

TECHNICAL ASSESSMENT

1

RANDOM WALK SIMULATION USING AGENTPY

Student Name	Larrion Luces
Section:	
Professor:	
Date:	

I. BACKGROUND INFORMATION

A **random walk** is a mathematical process that describes a path consisting of a sequence of **random steps**. Random walks are used in many fields, such as physics, biology, economics, and computer science, to model systems that evolve randomly over time.

In this laboratory, we simulate multiple agents moving randomly on a grid using **AgentPy**, a Python library for **agent-based modeling**. Each agent represents an entity that can move one step at a time in a random direction. This simulation helps students understand:

- How **agents move and interact** in a grid-based environment.
- The concept of **bounded movement**, ensuring agents stay within the grid limits.
- What changing **parameters** like the number of agents, grid size, or number of steps affects the system.
- How **agent-based models** can be used to simulate real-world phenomena, such as **diffusion, crowd dynamics, or swarm behavior**.

II. TECHNICAL ASSESSMENT

RANDOM WALK SIMULATION USING AGENTPY

Objective

At the end of this laboratory activity, students are expected to:

1. To understand the concept of **random walks** in a grid environment.
2. To implement a **multi-agent simulation** using Python and AgentPy.
3. To visualize the **movement of agents** on a grid over time.

Materials Needed

- Computer with Python 3.x installed
- Libraries: agentpy, matplotlib
- Code editor (e.g., VSCode, PyCharm, or Jupyter Notebook)

General Instructions

1. This **Technical Assessment (TA4)** must be completed individually.
2. Follow the Task below.
3. Include clear comments in your code to describe its logic.
4. Each program should display the correct input, process, and output.
5. Ensure that your script displays clear and readable outputs in the Command Window.
6. Compile all Python codes and corresponding outputs into one PDF file.
7. Save it as **TA1_lastname.pdf**
   (Example: TA1_Patilano.pdf)

8. Submit the **python file** and **PDF** file.

Activity Instructions

Step 1: Run the Code

1. Open the provided Python code.

Random Walk Simulation (Console-Based)

```
import agentpy as ap
import random
import matplotlib.pyplot as plt
import matplotlib.animation as animation

# -------- Get user input from console --------
num_agents = int(input("Enter number of agents: "))
grid_size = int(input("Enter grid size (e.g., 10 for 10x10): "))
num_steps = int(input("Enter number of steps: "))

# -------- Agent Definition -----------
class RandomWalker(ap.Agent):
    def step(self):
        direction = random.choice([(1,0), (-1,0), (0,1), (0,-1)])
        x, y = self.position
        # Keep agents inside grid
        x = max(0, min(self.model.p.grid_size[0]-1, x + direction[0]))
        y = max(0, min(self.model.p.grid_size[1]-1, y + direction[1]))
        self.position = (x, y)

# -------- Model Definition -----------
class RandomWalkModel(ap.Model):
    def setup(self):
        # Create agents
        self.agents = ap.AgentList(self, self.p.agents, RandomWalker)

        # Initialize random positions
        for agent in self.agents:
            agent.position = (random.randint(0, self.p.grid_size[0]-1),
                              random.randint(0, self.p.grid_size[1]-1))

        # Create grid (optional)
        self.grid = ap.Grid(self, self.p.grid_size, torus=False)
        self.grid.add_agents(self.agents)

    def step(self):
        for agent in self.agents:
            agent.step()

# -------- Parameters from user input -----------
parameters = {
    'agents': num_agents,
    'grid_size': (grid_size, grid_size),
    'steps': num_steps
}
```

```
# -------- Run Model -----------
model = RandomWalkModel(parameters)
model.setup()

# -------- Interactive Animation -----------
fig, ax = plt.subplots()
ax.set_xlim(0, model.p.grid_size[0])
ax.set_ylim(0, model.p.grid_size[1])
ax.set_xticks(range(model.p.grid_size[0]+1))
ax.set_yticks(range(model.p.grid_size[1]+1))
ax.grid(True)

scat = ax.scatter([], [], s=200)  # s = size of agents

def update(frame):
    model.step()
    x = [agent.position[0] for agent in model.agents]
    y = [agent.position[1] for agent in model.agents]
    scat.set_offsets(list(zip(x, y)))
    return scat,

ani = animation.FuncAnimation(fig, update, frames=model.p.steps, blit=True, repeat=False)
plt.show()
```

2. Run the program in your terminal.
3. Enter the following values when prompted:
    - **Number of agents** (e.g., 5)
    - **Grid size** (e.g., 10 for a 10x10 grid)
    - **Number of steps** (e.g., 20)

These values control the number of agents, the size of the environment, and the simulation duration.

Step 2: Understand the Agent Behavior

1. Each agent is represented by a dot in the grid.
2. Agents move **randomly** in one of four directions: up, down, left, or right.
3. Agents **cannot leave the grid**; movement is restricted within the grid boundaries.

Step 3: Observe the Simulation

1. After running, a **grid with moving agents** is displayed.
2. Each time step updates the position of all agents.
3. Observe:
    - How agents spread across the grid over time
    - Whether agents cluster in certain areas or move freely

Step 4: Experiment with Parameters

- Repeat the simulation with **different values**:
    - **More agents** (e.g., 10 or 20)
    - **Smaller/larger grid sizes**
    - **Longer simulations** (e.g., 50 steps)

- Observe and record how the **agent movement patterns change**.

Laboratory Tasks

1. **Track agent paths**: Modify the code to **record and display the path of each agent** during the simulation.
2. **Change movement rules**: Make agents **avoid each other** or move in a **preferred direction** and observe the resulting patterns.
3. **Analyze final positions**: Record the **final positions** of all agents and analyze their **distribution**.
4. **Compare scenarios**: Run **two simulations with different parameters** and compare the movement patterns.

Modified (Updated Code - Laboratory Tasks) ⇩

```
import agentpy as ap
import random
import matplotlib.pyplot as plt
import matplotlib.animation as animation
import numpy as np
from IPython.display import HTML, display

# -------- Get user input --------

num_agents = int(input("Enter number of agents: "))
grid_size = int(input("Enter grid size (e.g., 10 for 10x10): "))
num_steps = int(input("Enter number of steps: "))

if num_agents <= 0 or grid_size <= 0 or num_steps <= 0:
    raise ValueError("All inputs must be positive integers.")

if num_agents > grid_size ** 2:
    raise ValueError(
        "The number of agents cannot exceed the number of grid cells "
        "when collision avoidance is enabled."
    )

# -------- Agent Definition --------

class RandomWalker(ap.Agent):

    def setup(self):
        self.path = []

    def step(self):
```

```
        # Preferred movement toward the upper-right
        moves = [(1, 0), (-1, 0), (0, 1), (0, -1)]
        weights = [0.4, 0.1, 0.4, 0.1]
        direction = random.choices(moves, weights=weights, k=1)[0]

        x, y = self.model.grid.positions[self]

        new_x = max(
            0,
            min(self.model.p.grid_size[0] - 1, x + direction[0])
        )
        new_y = max(
            0,
            min(self.model.p.grid_size[1] - 1, y + direction[1])
        )

        new_position = (new_x, new_y)

        # Collision avoidance
        occupied_positions = {
            self.model.grid.positions[agent]
            for agent in self.model.agents
            if agent != self
        }

        if new_position not in occupied_positions:
            self.model.grid.move_to(self, new_position)

        self.path.append(self.model.grid.positions[self])

# -------- Model Definition --------

class RandomWalkModel(ap.Model):

    def setup(self):
        self.grid = ap.Grid(
            self,
            self.p.grid_size,
            torus=False,
            track_empty=True
        )

        self.agents = ap.AgentList(
            self,
```

```
            self.p.agents,
            RandomWalker
        )

        # Assign unique random starting positions
        self.grid.add_agents(
            self.agents,
            random=True,
            empty=True
        )

        for agent in self.agents:
            agent.path.append(self.grid.positions[agent])

    def step(self):
        # Random order reduces first-agent advantage
        agents_in_random_order = list(self.agents)
        random.shuffle(agents_in_random_order)

        for agent in agents_in_random_order:
            agent.step()

# --------  Parameters --------

parameters = {
    "agents": num_agents,
    "grid_size": (grid_size, grid_size),
    "steps": num_steps
}

# --------  Run Simulation --------

model = RandomWalkModel(parameters)

if not hasattr(model, "agents"):
    model.setup()

for _ in range(num_steps):
    model.step()

# --------  Create Animation from Recorded Paths --------

fig, ax = plt.subplots(figsize=(7, 7))
```

```
ax.set_xlim(-0.5, grid_size - 0.5)
ax.set_ylim(-0.5, grid_size - 0.5)
ax.set_xticks(range(grid_size))
ax.set_yticks(range(grid_size))
ax.set_title("Random Walk with Preferred Direction and Collision Avoidance")
ax.set_xlabel("X position")
ax.set_ylabel("Y position")
ax.grid(True)

colors = plt.cm.tab20(np.linspace(0, 1, num_agents))

initial_positions = np.array([
    agent.path[0]
    for agent in model.agents
])

scat = ax.scatter(
    initial_positions[:, 0],
    initial_positions[:, 1],
    s=150,
    c=colors,
    edgecolors="black",
    zorder=5
)

def update(frame):
    current_positions = [
        agent.path[frame]
        for agent in model.agents
    ]

    scat.set_offsets(np.array(current_positions))

    for i, agent in enumerate(model.agents):
        visible_path = agent.path[:frame + 1]
        path_x = [position[0] for position in visible_path]
        path_y = [position[1] for position in visible_path]
        lines[i].set_data(path_x, path_y)

    ax.set_title(
        f"Random Walk Simulation — Step {frame}/{num_steps}"
    )

    return [scat] + lines
```

```
ani = animation.FuncAnimation(
    fig,
    update,
    frames=range(num_steps + 1),
    interval=400,
    blit=False,
    repeat=False
)

display(HTML(ani.to_jshtml()))
plt.close(fig)

print("\n=== FINAL POSITION ANALYSIS ===")

final_positions = [
    model.grid.positions[agent]
    for agent in model.agents
]

for i, position in enumerate(final_positions, start=1):
    print(f"Agent {i} finished at: {position}")

x_coords = [position[0] for position in final_positions]
y_coords = [position[1] for position in final_positions]

print("\n--- Distribution Summary ---")
print(f"Average X position: {np.mean(x_coords):.2f}")
print(f"Average Y position: {np.mean(y_coords):.2f}")
print(f"Unique occupied cells: {len(set(final_positions))}")
print(f"Maximum coordinate: {grid_size - 1}")
```

Step 5: Answer the Questions

1. What happens if the number of agents is **very high** compared to the grid size?
2. How does increasing the number of steps affect the **spread of agents**?
3. Do the agents ever move outside the grid? Why or why not?
4. How can this simulation be used to **model real-life phenomena** (e.g., diffusion, crowd movement)?
5.

Answer the Questions ⇩

1. **If there is a high agent-to-grid ratio, the grid becomes overcrowded. If the collision rules are inactive, multiple agents will stack on top of each other, making it impossible to distinguish individuals. If collision avoidance is active, agents will experience "gridlock" or like a traffic jam, moving very slowly or getting entirely stuck because they have no empty adjacent spaces to step into.**

2. **In a purely random walk, increasing the number of steps increases the area of diffusion. Over a longer duration, the agents will disperse further from their starting points and cover a larger portion of the grid. If the agents have a preferred direction rule, increasing the steps will cause them to eventually bunch up tightly against the boundaries of that preferred direction.**

3. **No, they do not. This is due to the boundary constraint programmed into the agent's step function:**

```
x = max(0, min(self.model.p.grid_size[0]-1, x + direction[0]))
```

   The min and max functions clamp the agent's coordinates. If an agent tries to step beyond the maximum grid index, the min function forces it to stay at the border. If it tries to step below zero, the max function forces it to stay at 0.

4. **By adjusting the agent's behavioral rules, this base model translates into real-world situations. Using random walks can model diffusion, such as how gas particles disperse in a room or how a drop of dye spreads in water. If you introduce "preferred direction" rule and "collision avoidance", the simulation can accurately reflect crowd movement, like people navigating a narrow hallway or evacuating a stadium during an emergency. The model can be applied to epidemiology to stimulate the spread of a virus through a moving population simply tracking which agents share the same grid space and assigning them a "sick" status upon contact.**